Project Document

Introduction

Have you ever craved a specific takeaway and spent ages searching through restaurants, only to find out that they don't have the exact dish that you want? That's where **Pickier** comes in, a delivery app that filters by item to allow you to choose exactly what you want. This app targets takeaway lovers, but also caters to those with dietary requirements and restrictions. Users can compare specific items to highlight which contain common allergens or meet certain preferences (e.g. vegans/vegetarians).

Project aim: To build a prototype food delivery tool, enabling users to search, compare and order takeaway items by searching for a specific dish.

Project objectives:

- Implement a functional search tool that filters items within search radius.
- Allow item comparison, including allergens and calories.
- Enable basket and checkout process
- Develop a customer profile where user can see past orders

This report will outline the process of building this product. It will outline our initial specifications and requirements, discuss the implementation process and finally outline our testing strategy to evaluate how robust our prototype is.

Background

We benchmarked our product against food delivery sites like UberEats, Deliveroo, and JustEat, which allow users to search by cuisine and postcode. Similarly, **Pickier** lets users search by item name and postcode, returning restaurants within a 5 km radius (fixed to align with these sites). The comparison tool displays up to three items to avoid information overload, highlighting allergens, dietary requirements, and calories to enhance the user experience and differentiate our app from competitors.

Specifications and design

Requirements

The core functionality of Pickier is designed to allow users to search for takeaway items, compare options, manage a basket, and complete orders efficiently. We identified a set of technical and non-technical requirements, and within this divided our ideas into must-haves and nice-to-haves features.

Technical requirements:

Must-have features

- Search tool: Users can search for a specific dish against a database of items.
- Location mapping/filtering: Results show items within a set radius of the user's postcode using coordinates derived from the postcode (using an external API).
- Item comparison: Users can compare up to 3 items to inspect calories, allergens, dietary requirements, price, ratings and delivery time.
- Basket/shopping cart: Users can add selected items to a basket.
- Checkout: Users can complete orders efficiently.

Nice-to-have features:

- Customer profile: Users can save personal information and view past orders. This could give users an option for faster checkout.
- Post-order management: Users can track their orders and receive estimated delivery times.
- Post-order reviews: Users can leave feedback on completed orders.
- AI chatbot/recommendation tool: The tool can provide personalised suggestions based on user preferences and search history. The chatbot could also provide assistance during and after ordering.

Non-technical requirements

- Performance: Search results and comparisons should load quickly for a smooth user experience.
- Usability:The interface should be visually appealing, easy to navigate and accessible. We didn't want the user to feel 'trapped' at any point in the app.
- Consistency: The styling and layout should remain uniform across all pages and components.
- Scalability and maintainability: The system architecture should allow team collaboration and future feature development.
- Responsiveness: Search and filtering should feel responsive even with our limited dataset. Users should always get feedback from clicking on or using a component.

Design and architecture

Once we agreed on the core features, we mapped the user journey and created wireframes to visualise how each screen should look and behave. This helped us to organise our approach before starting development.

System architecture

1. Frontend (React.js)
    - Handles UI, navigation and state management
    - Composed of pages representing each of our key features
    - Features contain components (such as buttons, search bars etc.) which aid the functioning of the pages
2. Backend (Node.js + Express REST API)
    - Handles requests from the frontend, communicating with the database and external APIs

    - Uses routes to GET and POST information between the client side, server and database (e.g. GET items for item searching or POST orders after order completion)
    - Calls an external API to help with the location mapping
3. Database (MySQL)
    - Stores item, allergen, restaurant, customer and order details

External services:

- Postcodes.io API: Geolocation tool used in our location mapping feature
- AI tools: Used for creating data to populate our database with

User Interface Design

This section contains images - for conciseness they have been linked as separate documents (they are also available in our repo)

- Our wireframes map the proposed design for our tool
- The main user flow is: Search → Results → Comparison → Basket → Checkout
    - Our user flow diagram shows the proposed user flow for our tool
- We wanted to create a clear layout with feature colours to make our app memorable and distinctive from its competitors. We chose to use a background colour that was off white and is known to be more accessible for people with dyslexia, as it reduces glare and visual stress while making text easier to read and follow.

Implementation and execution

Development approach

After agreeing on a project idea, we created a wireframe to map the user journey and identify required backend tools and database structure. From this, we defined must-have and nice-to-have features, delegating core features to team members. We built the MVP iteratively, linking features through close communication, then refined them and added tests to ensure stability. Basic styling was applied during development, with final design adjustments made after integration for consistency

We held check-in meetings at least once per week to track progress, where we discussed our current status, aims for the next week and blockers. For day-to-day communication we used our Slack channel and huddle meetings for team members who needed to work collaboratively. We also documented meeting notes for reference and to support members who were unable to attend any set meetings. The team used GitHub effectively, submitting pull requests for all code changes and ensuring reviewers were assigned for reviews to be completed before merging.

Tools and libraries used

- The application was developed using a modern full-stack toolset. The frontend was built with **React** and **React Router**, using **Material UI** for consistent, accessible

styling and **Axios/Fetch** for API communication. Component testing was carried out using **Jest** and **React Testing Library**.

- The backend was implemented as a **Node.js and Express** RESTful API, connected to a **MySQL** database for storing restaurant, item, allergen, and order data. Environment variables were managed using **dotenv**, and middleware such as **CORS** was used to support secure client–server communication. The **Postcodes.io API** was integrated for postcode-to-coordinate conversion.
- Development and collaboration were managed using **Git and GitHub** for version control, **VS Code** as the primary IDE, and **Postman** for testing backend endpoints.

Implementation process

Achievements

One of the key achievements of this project was our ability to make important decisions quickly. Working in a group with members who had different availability and commitments was a challenge. However, we were able to converge on an idea early on, which allowed us to start ideation and develop a clear vision of what we wanted to build. The chosen idea was relatable to everyone on the team, which proved beneficial when designing features, as we each had a reference point from similar apps. This clarity made task breakdown and delegation much easier, as we could identify what needed to be built and understand how each component fit into the overall flow of the app. It also facilitated effective task delegation, as everyone had a general understanding of each feature's purpose and function. Having a strong understanding of the project goal was able to keep us aligned on what action steps needed to be taken throughout.

A key achievement in this project was database design. We were aware that the right data was integral to the success of this project. We designed a SQL database comprising the elements required for our app functionality, aiming to adhere to good database standards and minimise redundancy. We were able to use AI to populate our database, which wasn't as straightforward as hoped, but did save considerable time. We set up multiple routes to allow the frontend to access and push data to the database, which resulted in smooth running of the tool and efficient linking of many of our features. Creating our own database (rather than relying on public APIs) enabled our app to run according to our expectations, as it allowed users to compare specific food items including their ratings, price, calories, and other details, to make informed decisions.

Another achievement was the successful implementation of a shared basket feature using stored context. A key feature of any e-commerce platform is the storage of basket items as the user moves through pages. We needed to consider this to improve the user experience, and to allow a smooth transition between the basket and the checkout. We utilised a global store to manage the state of the basket globally, allowing users to add, remove and update items while navigating between different pages of the application, without losing their selections. This was one of the more challenging parts of building our tool, so it was a huge achievement when we got it working.

Challenges

As this was our first experience building a full React application and collaborating on a shared Git repository, we encountered several technical and non-technical challenges. Technically, we struggled with Git conflicts, feature integration points, implementing location mapping, and using AI to populate our database. From a non-technical perspective, learning new tools while actively developing the application required strong communication and coordination, which occasionally contributed to version control issues.

Location mapping was a core requirement for the app. While we would have liked to implement a full mapping tool like open street maps of Google Maps, we went for a simpler solution, using a postcode-to-coordinates API combined with a distance calculation function to measure as the crow flies distance in km. For the MVP, we built item-searching only to link to the comparison page. Combining this with the location search was a challenge. Managing the asynchronous postcode API calls required restructuring our search logic within the userEffect hook, which was difficult but ultimately successful. With more time, we would implement route based mapping for improved location searching, allowing for prediction of delivery time estimates.

Using AI (Gemini) to populate the SQL database also presented challenges. Although this approach saved time, the generated data was often inaccurate, particularly with postcodes, requiring multiple rounds of queries to get the correct data. Even so, many of the postcodes in the database do not exist, therefore searching for an exact restaurant using their postcode will often not work. This does not impact the functioning of the app, so we decided not to spend more time troubleshooting. While functional for a prototype, this data would need to be replaced with real restaurant data in a production system.

Finally, repository organisation was a recurring challenge. Early in development, an undefined folder structure led to frequent merge conflicts and restructuring. These issues were resolved through in depth team discussions, standardising the project structure, and separating API routes from the main server file. Although these changes temporarily disrupted development, they significantly improved code organisation and team workflow, and provided valuable lessons for future projects.

Changes Made

After our feedback session, we discussed the location of our ‘About Us’ page within our project and focused on ensuring this had a natural flow, as this was currently set up to follow on from the ‘Checkout’ page. After consideration, we realised this did not reflect the natural flow for a user journey when checking out on a page and decided to create a navbar with routing so that the user could select this navigation instead of it being a built-in part of the journey. This was also nice as it reflected an ‘About Us’ page that you can find on other company websites.

We decided to add routing to our app as we wanted there to be some flexibility for the user. For example, the user may have started searching for their product and added this to the cart, but may have wanted to return to the homepage to search for something else also. The

user may also want to view their customer profile and amend any information without adding anything to their cart, or simply just view what's currently inside their basket.

Agile development

We focussed on building a functional tool, rather than a perfect one. From the outset, we aimed to build an MVP, demonstrating the flow and routing through the tool, before developing the features further. This has been discussed above, regarding how we developed the item search tool. We also used refactoring as we worked. For example, we initially started out with one server.js file, however when this started getting too long and confusing, we used express.router to break the files into separate routes, this made our code more readable and easier to debug. We implemented a review process when merging anything in GitHub and ensured that pull requests were not merged without review from another group member, this helped to keep our code high quality and was a valuable time for feedback in the iterative development process.

Testing and evaluation

Testing strategy

Our goal wasn't just to write code that runs, but to build an application that feels stable and trustworthy for the user. To achieve this, we adopted a Component-Driven approach. This meant we didn't try to test the whole app at once; instead, we broke it down and tested individual pieces (like the Basket logic or the Comparison Table) to make sure they worked in isolation before putting them together.

Tools Used

- **Jest:** We used this as our main "calculator." It was perfect for testing the logic behind the scenes specifically, checking that our distance algorithms (Haversine formula) were accurate and that the shopping basket added up the totals correctly.
- **React Testing Library (RTL):** We used this to simulate how a real human interacts with the site. Instead of looking at the code, RTL helped us answer questions like: "When I click 'Add to Cart', does the button actually work?" and "Does the error message show up if I type a fake postcode?"
- **Functional user testing**
  Beyond the automated code tests, we manually ran through the application acting as if we were real customers. We focused on the "Happy Path": the ideal journey a user takes from searching for food to checking out while also trying to "break" the app by entering wrong data to see if it handled errors gracefully.

System limitations

We feel that our app successfully meets the core requirements of the MVP that we set out to achieve for this project. However there are several limitations of the tool that we were not able to address in the given timeframe:

- Data accuracy and real-time updates: Our tool uses a static database, which relies on manual updates. There is no integration with third party APIs, therefore all the

data is fake and was generated and input by us. In the future, this database system would be replaced by real data from restaurants, input using an API and verified using inbuilt verification tools.

- Geographical constraints: Currently, the database is only built to contain restaurants from London, due to limitations with generating a large amount of AI generated data. As mentioned earlier, the distance calculation is performed as the crow flies, so is not accurate actual location mapping. This would need improving within the prototype before the product could be used in the real world, we would ideally integrate a tool like Google Maps to take care of our mapping needs.
- Technical and scalability: Our app is currently not fit for roll out for mass use, its current infrastructure would not perform well if thousands of users attempted to compare items simultaneously. The app requires a new payment processing feature, and the checkout needs to be revised to handle multiple orders. This would be the next stage for development of this product if we had more time.
- Platform limitations: The tool is currently only functional for desktop and standard mobile browsers. For a commercial use, a mobile app would need to be developed to provide a competitive user experience.

Conclusion

In this project, we successfully developed a working prototype of our food delivery app, Pickier, designed to meet the needs of picky eaters and users with dietary requirements. The application delivers a complete end-to-end user journey, enabling users to search for food items, compare options across restaurants, manage a basket, complete a simple checkout process and access a customer profile. By using tools such as React, Node.js, MySQL and external APIs, we have demonstrated the feasibility of an item-first food ordering platform and delivered an interactive MVP.

While the prototype meets the core MVP requirements, several improvements could be made in future iterations. These include integrating real restaurant data, implementing map-based routing, enabling mobile responsiveness and adding secure, third party payment processing. Additional features such as order tracking and user reviews could further enhance the user experience.

Overall, this project provided valuable hands-on experience in full-stack development, from system design to implementation. We have all learned a lot during the development of this project, from a technical perspective, as our first time building a functional product, but also non-technical skills such as the importance of clear communication and effective teamwork.